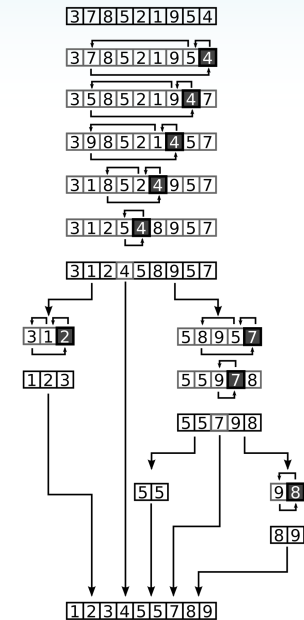


6.506: Algorithm Engineering

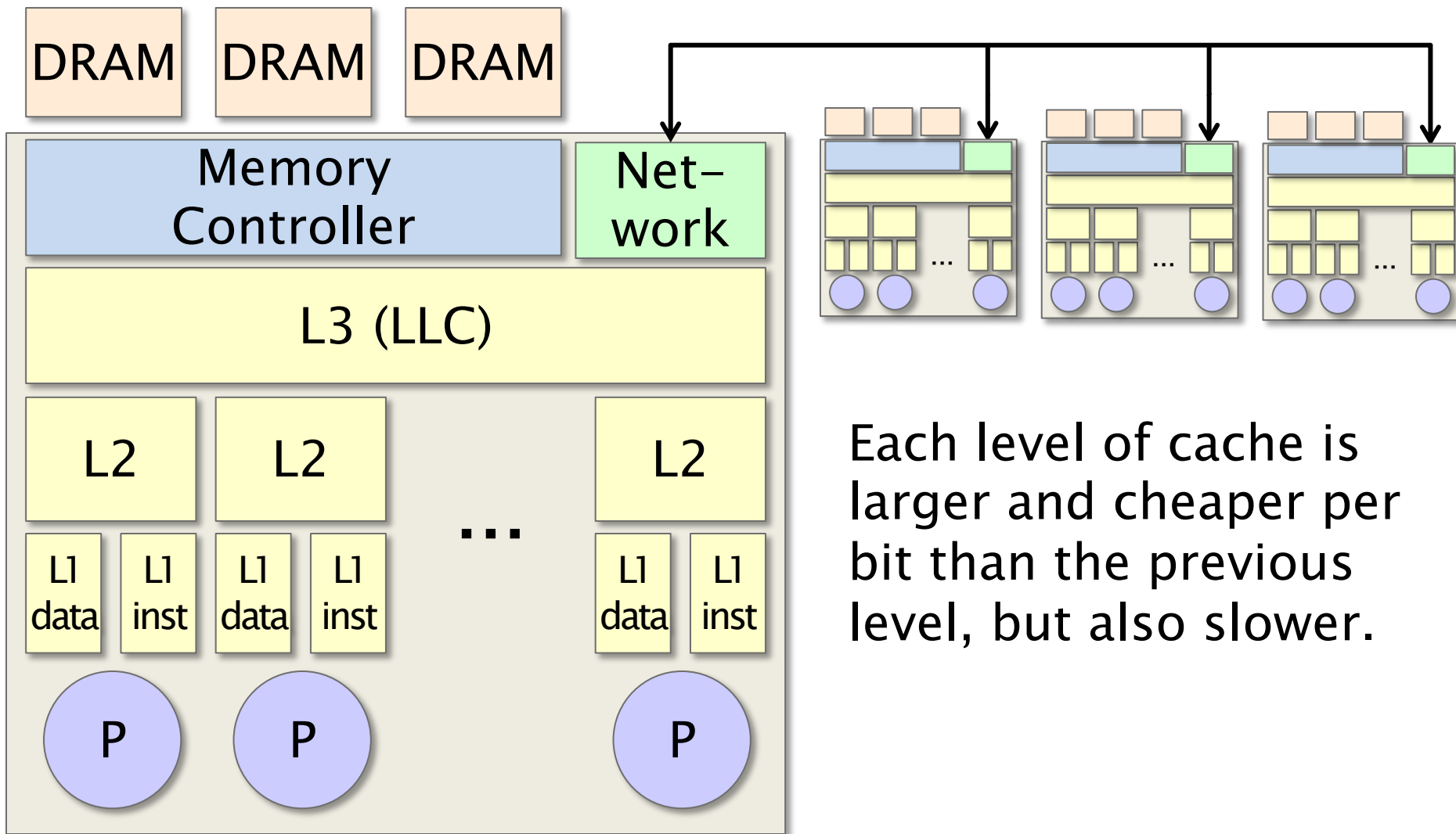


LECTURE 3 CACHE-OBLIVIOUS ALGORITHMS AND DATA STRUCTURES

Charles E. Leiserson
February 14, 2023



Multicore Cache Hierarchy



Cache Specs for Typical High-End Multicore

Level	Size/core	Associativity	Latency (cycles)
DRAM	up to 160 GiB		85–240
L3	1.375 MiB	11	50–70
L2	1 MiB	16	14
L1–D	32 KiB	8	4–5
L1–I	32 KiB	8	5

Intel Xeon Platinum 8280L (Cascade Lake)

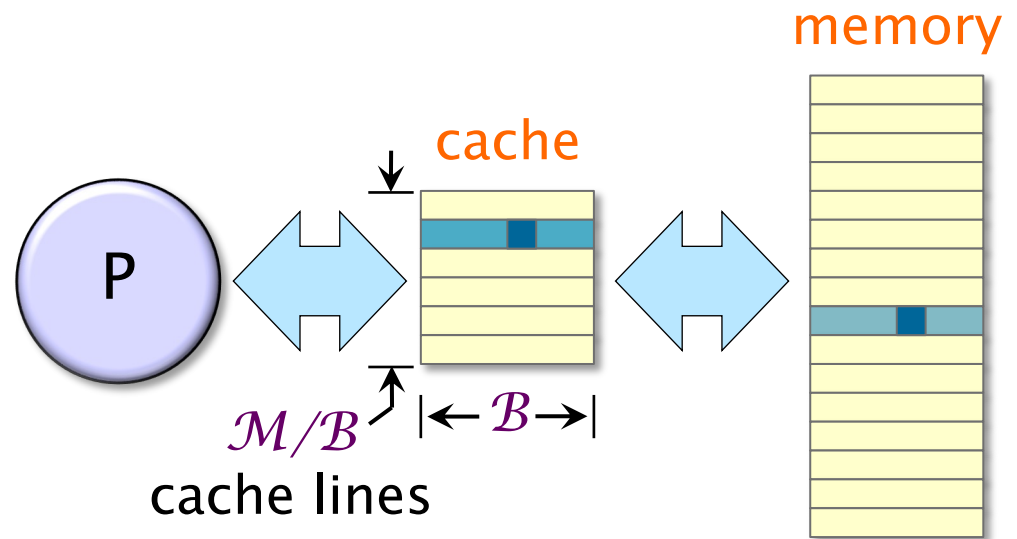
- Launched April 2019 for \$17,906 — cheaper now.
- 2.7 GHz clock, Turbo Boost up to 4 GHz
- 28 cores/chip + 2-way hyperthreading
- 2190 GFLOPS
- 64 B cache lines/blocks
- Up to 8-way multiprocessing

So, if our processor has 2.7 GHz clock speed, then
1 cycle = $1 / (2.7 \times 10^9) = 0.37 \text{ ns}$
5 cycles = $5 \times 0.37 \text{ ns} = 1.85 \text{ ns}$

Ideal-Cache Model

Parameters

- Two-level hierarchy.
- Cache size of $\mathcal{M}$ bytes.
- Cache-line length of $\mathcal{B}$ bytes.
- Fully associative.
- Optimal, omniscient replacement.

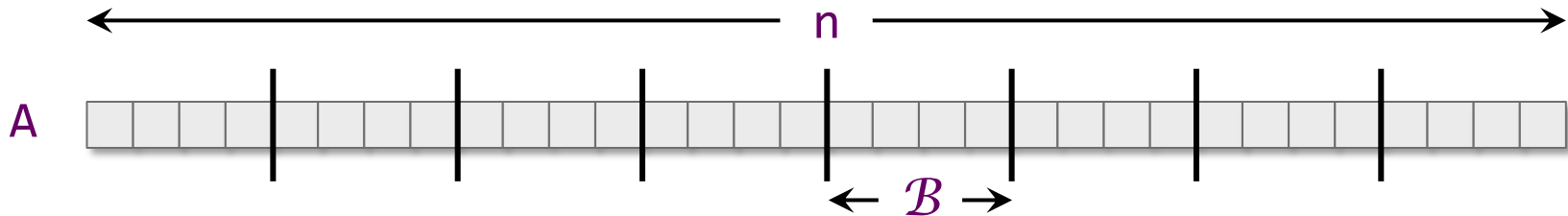


Performance Measures

- **work** T (ordinary running time)
- **cache misses** Q

Reading an Array Sequentially

```
sum = 0;  
for (int i=0; i<n; ++i)  
    sum += A[i];
```



Cache misses: $Q(n) = \Theta(n/B)$

Segment Caching Lemma

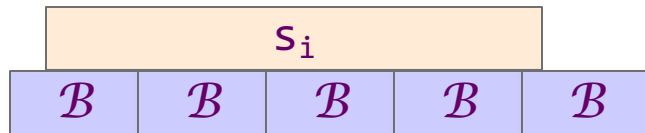
Lemma. Suppose that a program reads a set of r data segments, where the i th segment consists of s_i contiguous bytes in memory, and suppose that

$$\sum_{i=1}^r s_i = N < \mathcal{M}/3 \text{ and } N/r \geq \mathcal{B}.$$

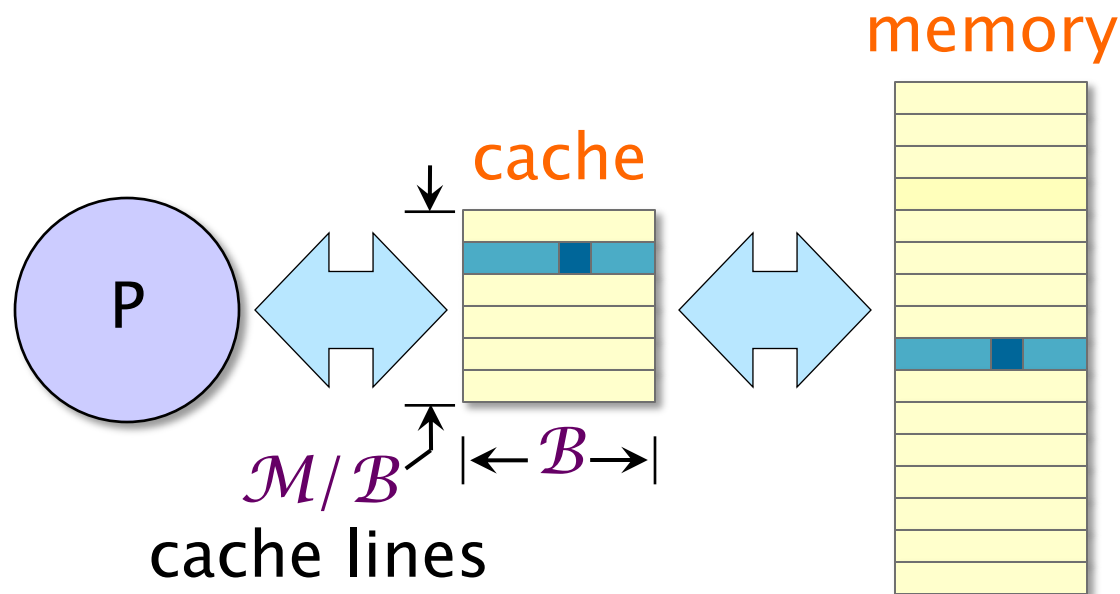
Then all the segments fit into cache, and the number of misses to read them all is at most $3N/\mathcal{B}$.

Proof. A single segment s_i incurs at most $s_i/\mathcal{B} + 2$ misses, and hence we have

$$\begin{aligned} \sum_{i=1}^r (s_i/\mathcal{B} + 2) &= N/\mathcal{B} + 2r \\ &= N/\mathcal{B} + (2r\mathcal{B})/\mathcal{B} \\ &\leq N/\mathcal{B} + 2N/\mathcal{B} \\ &= 3N/\mathcal{B}. \quad \blacksquare \end{aligned}$$



Tall Caches



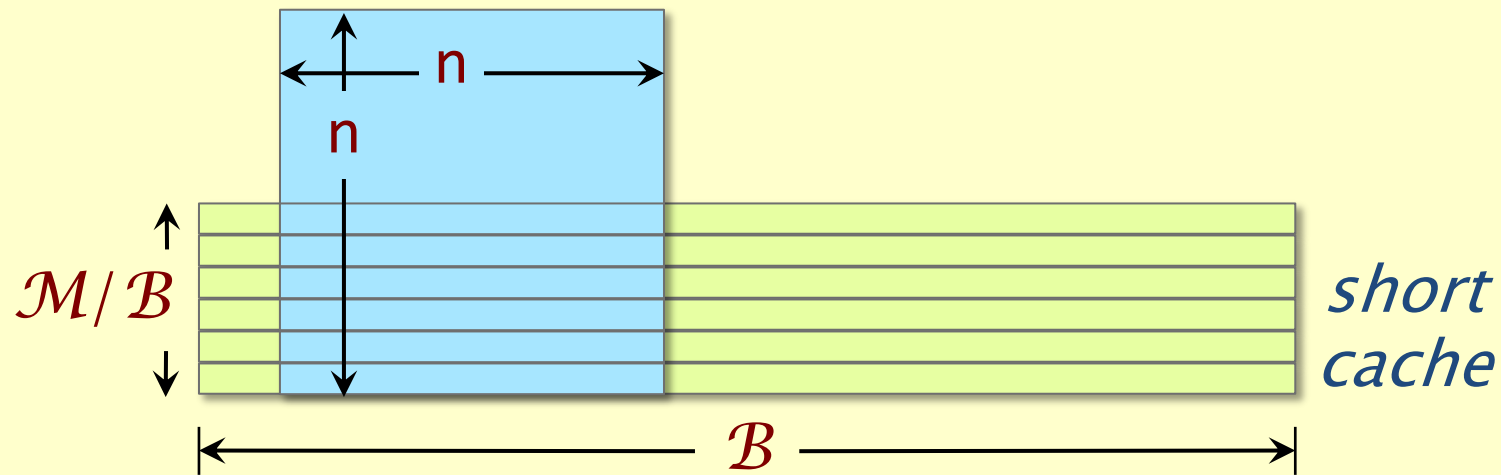
Tall-cache assumption

$B^2 < c \mathcal{M}$ for some sufficiently small constant $c \leq 1$.

Example: Intel Xeon Platinum 8280L

- Cache-line length $B = 64$ bytes.
- L1-cache size $\mathcal{M} = 32$ kibibytes.

What's Wrong with Short Caches?

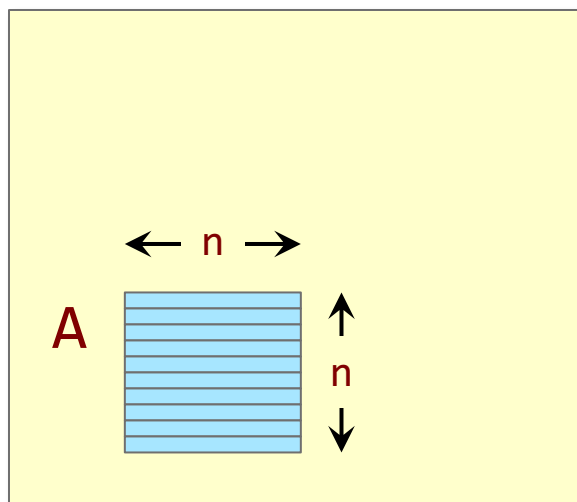


Tall-cache assumption

$B^2 < cM$ for some sufficiently small constant $c \leq 1$.

An $n \times n$ submatrix stored in row-major order may not fit in a short cache even if $n^2 < cM$!

Submatrix Caching Lemma

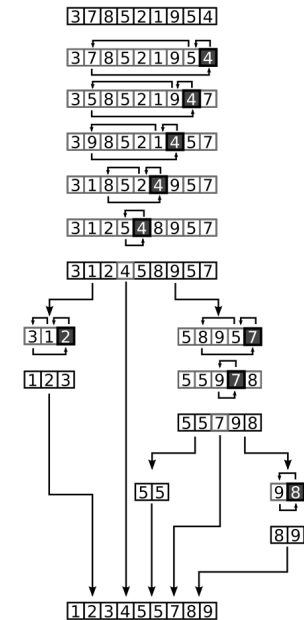


Lemma. Suppose that an $n \times n$ submatrix A is read into a tall cache satisfying $\mathcal{B}^2 < c\mathcal{M}$, where $c < 1/3$ is constant, and suppose that $c\mathcal{M} \leq n^2 < \mathcal{M}/3$. Then A fits into the cache, and the number of misses to read all of A 's elements is at most $3n^2/\mathcal{B}$.

Proof. We have $r = n$, $s_i = n$, $N = n^2$. Since $\mathcal{B}^2 < c\mathcal{M} \leq n^2$, we have $\mathcal{B} \leq n = N/r$. And since $N < \mathcal{M}/3$, the segment caching lemma applies. ■



CACHE ANALYSIS OF MATRIX MULTIPLICATION



Multiply Square Matrices

```
void Mult(double *C, double *A, double *B, int64_t n) {  
    for (int64_t i=0; i < n; i++)  
        for (int64_t j=0; j < n; j++)  
            for (int64_t k=0; k < n; k++)  
                C[i*n+j] += A[i*n+k] * B[k*n+j];  
}
```

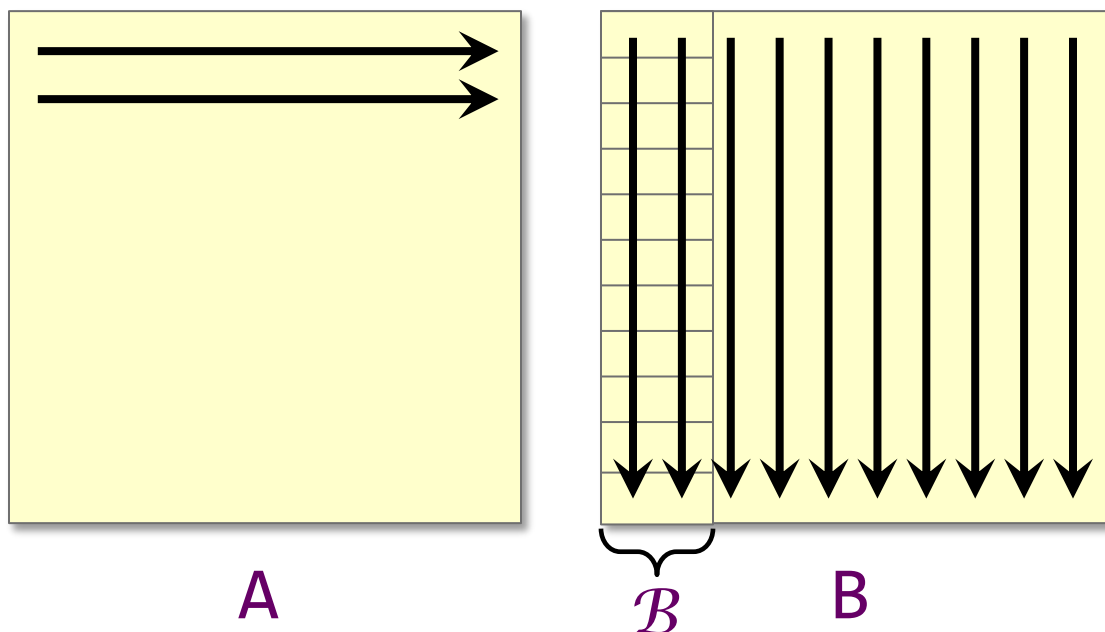
Analysis of work

$$T(n) = \Theta(n^3).$$

Analysis of Cache Misses

```
void Mult(double *C, double *A, double *B, int64_t n) {  
    for (int64_t i=0; i < n; i++)  
        for (int64_t j=0; j < n; j++)  
            for (int64_t k=0; k < n; k++)  
                C[i*n+j] += A[i*n+k] * B[k*n+j];  
}
```

Assume row major and tall cache



Case 1

$n \geq \mathcal{M}/\mathcal{B}$.

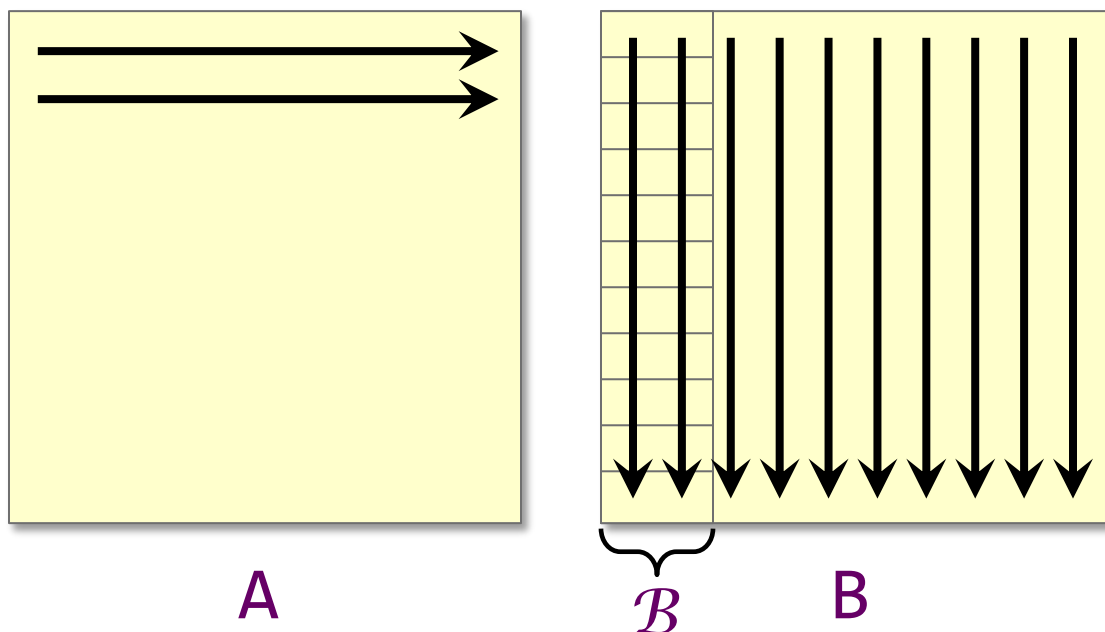
Analyze matrix **B**.
Assume LRU.

$Q(n) = \Theta(n^3)$, since
matrix **B** misses
on every access.

Analysis of Cache Misses

```
void Mult(double *C, double *A, double *B, int64_t n) {  
    for (int64_t i=0; i < n; i++)  
        for (int64_t j=0; j < n; j++)  
            for (int64_t k=0; k < n; k++)  
                C[i*n+j] += A[i*n+k] * B[k*n+j];  
}
```

Assume row major and tall cache



Case 2

$$\mathcal{M}^{1/2} \leq n < \mathcal{M}/\mathcal{B}.$$

Analyze matrix $\mathbf{B}$.

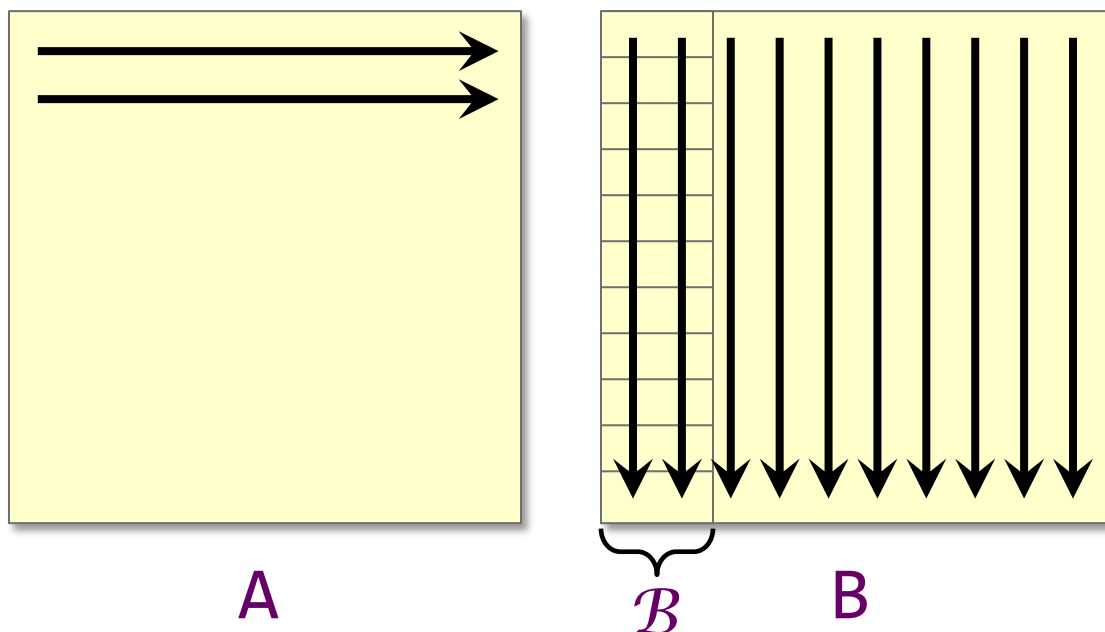
Assume LRU.

$Q(n) = n \cdot \Theta(n^2/\mathcal{B}) = \Theta(n^3/\mathcal{B})$, since matrix $\mathbf{B}$ can exploit spatial locality.

Analysis of Cache Misses

```
void Mult(double *C, double *A, double *B, int64_t n) {  
    for (int64_t i=0; i < n; i++)  
        for (int64_t j=0; j < n; j++)  
            for (int64_t k=0; k < n; k++)  
                C[i*n+j] += A[i*n+k] * B[k*n+j];  
}
```

Assume row major and tall cache



Case 3

$n < c\mathcal{M}^{1/2}$.

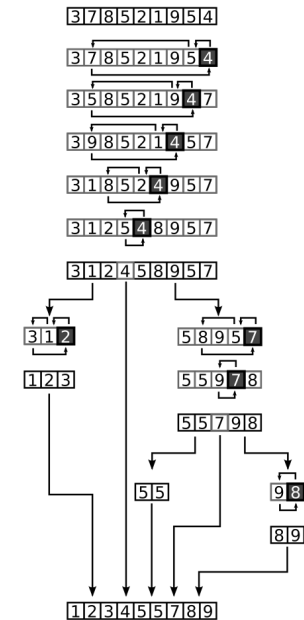
Analyze matrix **B**.

Assume LRU.

$Q(n) = \Theta(n^2/\mathcal{B})$, by
the submatrix
caching lemma.



DIVIDE & CONQUER



Recursive Matrix Multiplication

Divide-and-conquer on $n \times n$ matrices.

$$\begin{array}{|c|c|} \hline C_{11} & C_{12} \\ \hline C_{21} & C_{22} \\ \hline \end{array} = \begin{array}{|c|c|} \hline A_{11} & A_{12} \\ \hline A_{21} & A_{22} \\ \hline \end{array} \times \begin{array}{|c|c|} \hline B_{11} & B_{12} \\ \hline B_{21} & B_{22} \\ \hline \end{array}$$

$$= \begin{array}{|c|c|} \hline A_{11}B_{11} & A_{11}B_{12} \\ \hline A_{21}B_{11} & A_{21}B_{12} \\ \hline \end{array} + \begin{array}{|c|c|} \hline A_{12}B_{21} & A_{12}B_{22} \\ \hline A_{22}B_{21} & A_{22}B_{22} \\ \hline \end{array}$$

8 multiply-adds of $(n/2) \times (n/2)$ matrices.

Recursive Code

```
// Assume that n is an exact power of 2.
void Rec_Mult(double *C, double *A, double *B,
              int64_t n, int64_t rowsize) {
    if (n == 1)
        C[0] += A[0] * B[0];
    else {
        int64_t d11 = 0;
        int64_t d12 = n/2;
        int64_t d21 = (n/2) * rowsize;
        int64_t d22 = (n/2) * (rowsize+1);

        Rec_Mult(C+d11, A+d11, B+d11, n/2, rowsize);
        Rec_Mult(C+d11, A+d12, B+d21, n/2, rowsize);
        Rec_Mult(C+d12, A+d11, B+d12, n/2, rowsize);
        Rec_Mult(C+d12, A+d12, B+d22, n/2, rowsize);
        Rec_Mult(C+d21, A+d21, B+d11, n/2, rowsize);
        Rec_Mult(C+d21, A+d22, B+d21, n/2, rowsize);
        Rec_Mult(C+d22, A+d21, B+d12, n/2, rowsize);
        Rec_Mult(C+d22, A+d22, B+d22, n/2, rowsize);
    }
}
```

Coarsen base case to overcome function-call overheads.

Recursive Code

```
// Assume that n is an exact power of 2.  
void Rec_Mult(double *C, double *A, double *B,  
              int64_t n, int64_t rowsize) {
```

```
    if (n == 1)
```

```
        C[0] += A[0] * B[0];
```

```
    else {
```

```
        ➡ int64_t d11 = 0;
```

```
        ➡ int64_t d12 = n/2;
```

```
        ➡ int64_t d21 = (n/2) * rowsize;
```

```
        ➡ int64_t d22 = (n/2) * (rowsize+1);
```

```
        Rec_Mult(C+d11, A+d11, B+d11, n/2, rowsize);
```

```
        Rec_Mult(C+d11, A+d12, B+d21, n/2, rowsize);
```

```
        Rec_Mult(C+d12, A+d11, B+d12, n/2, rowsize);
```

```
        Rec_Mult(C+d12, A+d12, B+d22, n/2, rowsize);
```

```
        Rec_Mult(C+d21, A+d21, B+d11, n/2, rowsize);
```

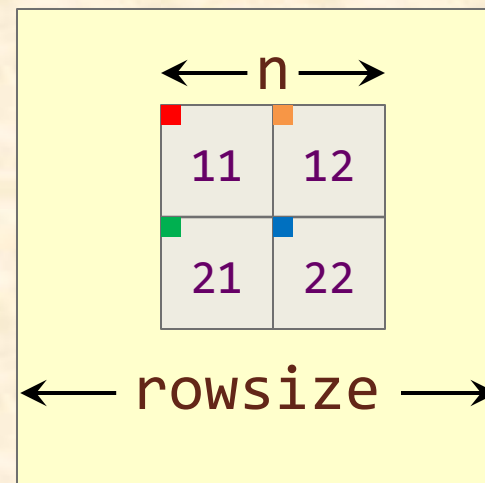
```
        Rec_Mult(C+d21, A+d22, B+d21, n/2, rowsize);
```

```
        Rec_Mult(C+d22, A+d21, B+d12, n/2, rowsize);
```

```
        Rec_Mult(C+d22, A+d22, B+d22, n/2, rowsize);
```

```
    }
```

```
}
```



Analysis of Work

```
// Assume that n is an exact power of 2.
void Rec_Mult(double *C, double *A, double *B,
              int64_t n, int64_t rowsize) {
    if (n == 1)
        C[0] += A[0] * B[0];
    else {
        int64_t d11 = 0;
        int64_t d12 = n/2;
        int64_t d21 = (n/2) * rowsize;
        int64_t d22 = (n/2) * (rowsize+1);

        Rec_Mult(C+d11, A+d11, B+d11, n/2, rowsize);
        Rec_Mult(C+d11, A+d12, B+d21, n/2, rowsize);
        Rec_Mult(C+d12, A+d11, B+d12, n/2, rowsize);
        Rec_Mult(C+d12, A+d12, B+d22, n/2, rowsize);
        Rec_Mult(C+d21, A+d21, B+d11, n/2, rowsize);
        Rec_Mult(C+d21, A+d22, B+d21, n/2, rowsize);
        Rec_Mult(C+d22, A+d21, B+d12, n/2, rowsize);
        Rec_Mult(C+d22, A+d22, B+d22, n/2, rowsize);
    }
}
```

$$\begin{aligned} T(n) &= 8T(n/2) + \Theta(1) \\ &= \Theta(n^3) \end{aligned}$$

Analysis of Work

$$T(n) = 8T(n/2) + 1$$

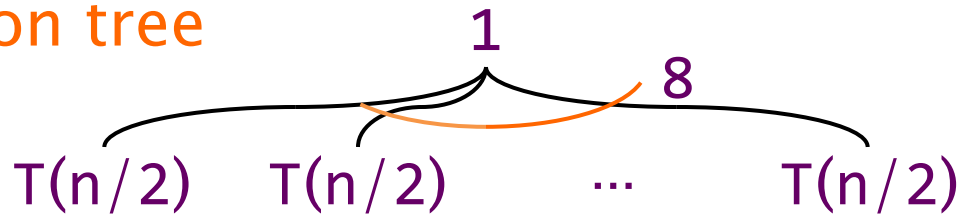
recursion tree

$T(n)$

Analysis of Work

$$T(n) = 8T(n/2) + 1$$

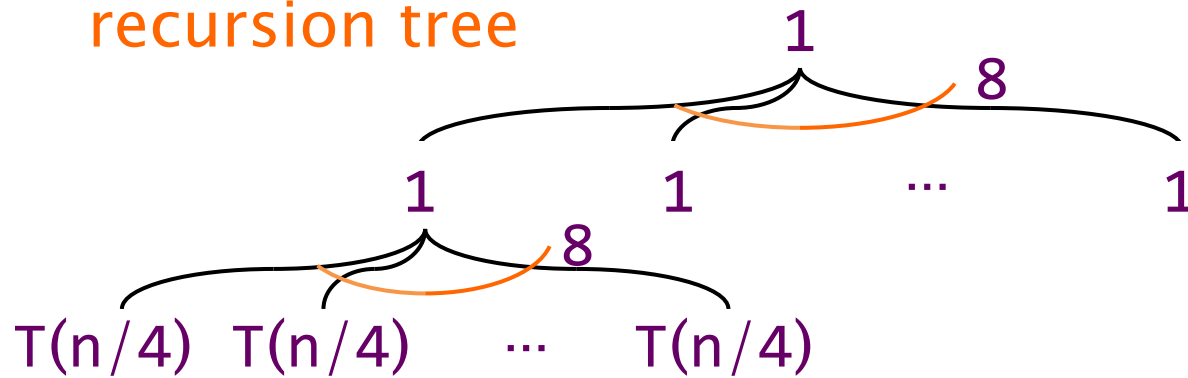
recursion tree



Analysis of Work

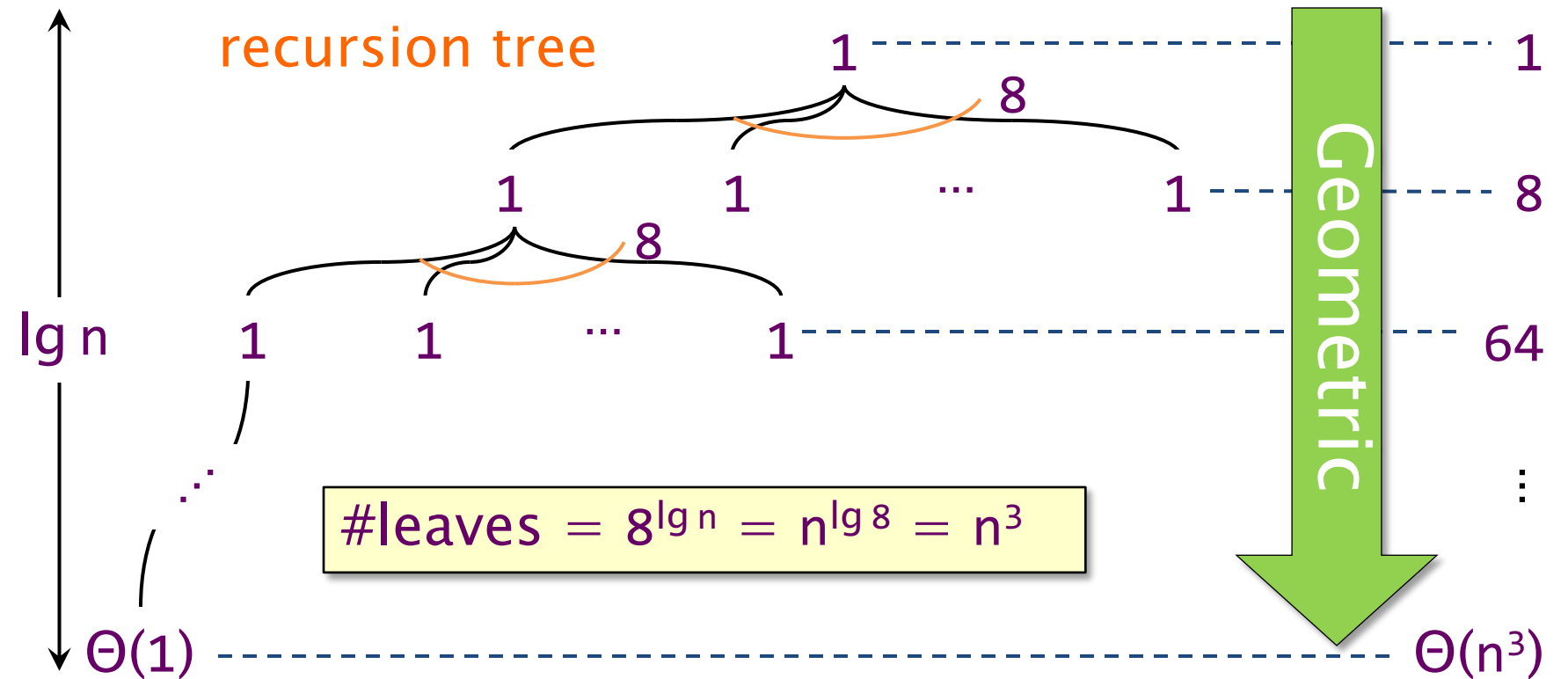
$$T(n) = 8T(n/2) + 1$$

recursion tree



Analysis of Work

$$T(n) = 8T(n/2) + 1$$



Note: Same work as looping versions.

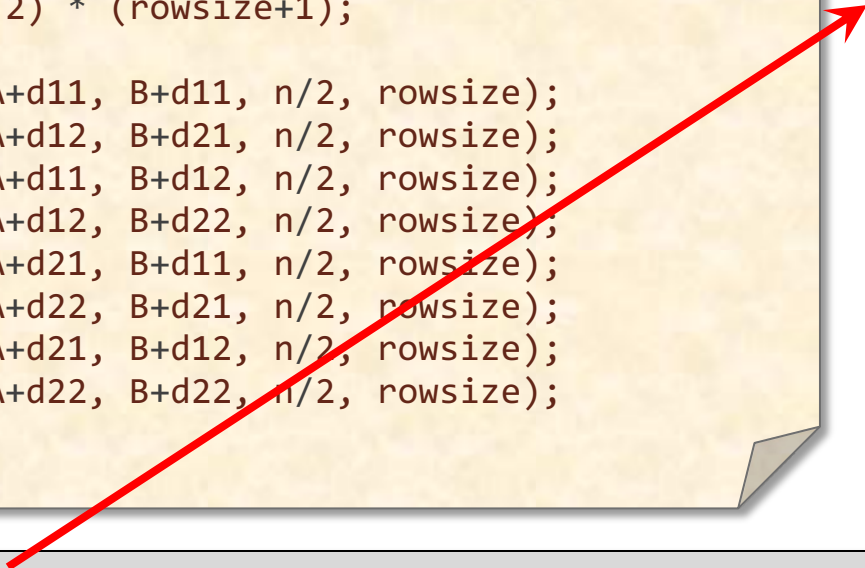
$$T(n) = \Theta(n^3)$$

Analysis of Cache Misses

```
// Assume that n is an exact power of 2.
void Rec_Mult(double *C, double *A, double *B,
              int64_t n, int64_t rowsize) {
    if (n == 1)
        C[0] += A[0] * B[0];
    else {
        int64_t d11 = 0;
        int64_t d12 = n/2;
        int64_t d21 = (n/2) * rowsize;
        int64_t d22 = (n/2) * (rowsize+1);

        Rec_Mult(C+d11, A+d11, B+d11, n/2, rowsize);
        Rec_Mult(C+d11, A+d12, B+d21, n/2, rowsize);
        Rec_Mult(C+d12, A+d11, B+d12, n/2, rowsize);
        Rec_Mult(C+d12, A+d12, B+d22, n/2, rowsize);
        Rec_Mult(C+d21, A+d21, B+d11, n/2, rowsize);
        Rec_Mult(C+d21, A+d22, B+d21, n/2, rowsize);
        Rec_Mult(C+d22, A+d21, B+d12, n/2, rowsize);
        Rec_Mult(C+d22, A+d22, B+d22, n/2, rowsize);
    }
}
```

Submatrix
caching
lemma



$$Q(n) = \begin{cases} \Theta(n^2/\mathcal{B}) & \text{if } n^2 < c\mathcal{M} \text{ for suff. small const } c \leq 1, \\ 8Q(n/2) + \Theta(1) & \text{otherwise.} \end{cases}$$

Analysis of Cache Misses

$$Q(n) = \begin{cases} \Theta(n^2/\mathcal{B}) & \text{if } n^2 < c\mathcal{M} \text{ for suff. small const } c \leq 1, \\ 8Q(n/2) + 1 & \text{otherwise.} \end{cases}$$

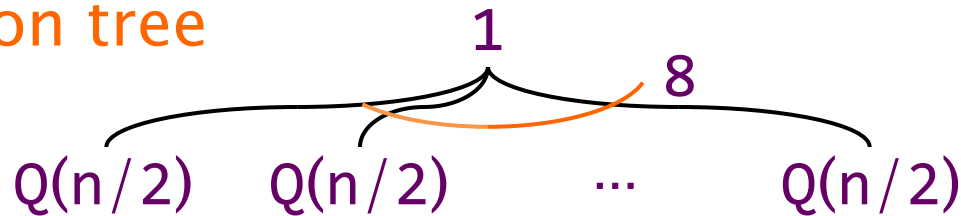
recursion tree

$Q(n)$

Analysis of Cache Misses

$$Q(n) = \begin{cases} \Theta(n^2/\mathcal{B}) & \text{if } n^2 < c\mathcal{M} \text{ for suff. small const } c \leq 1, \\ 8Q(n/2) + 1 & \text{otherwise.} \end{cases}$$

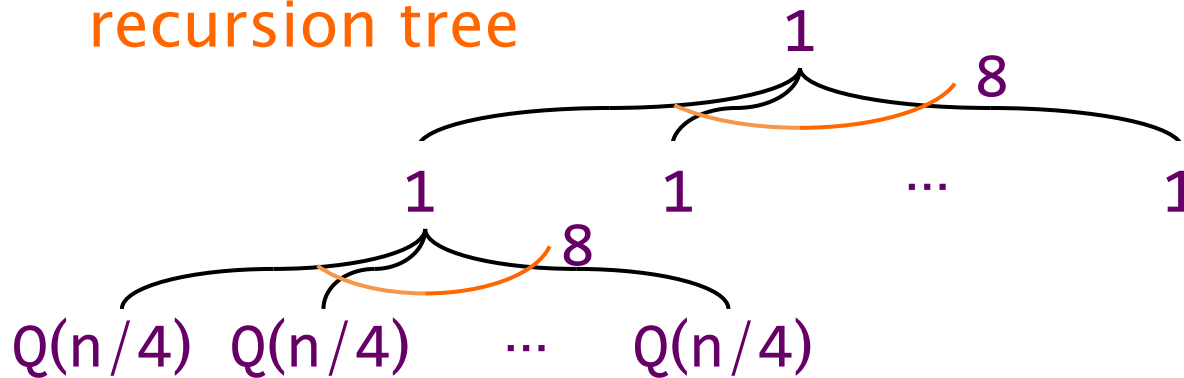
recursion tree



Analysis of Cache Misses

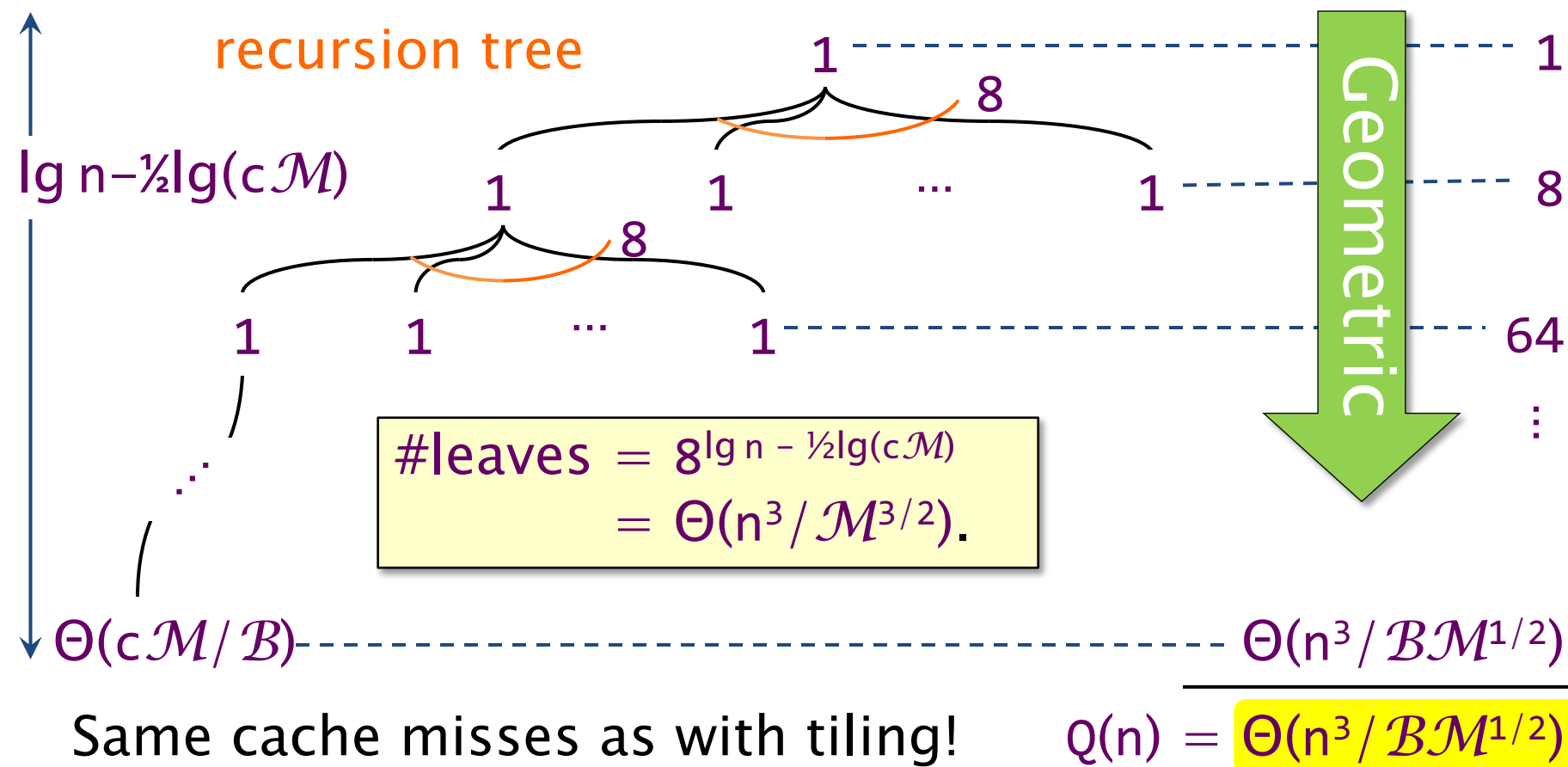
$$Q(n) = \begin{cases} \Theta(n^2/\mathcal{B}) & \text{if } n^2 < c\mathcal{M} \text{ for suff. small const } c \leq 1, \\ 8Q(n/2) + 1 & \text{otherwise.} \end{cases}$$

recursion tree



Analysis of Cache Misses

$$Q(n) = \begin{cases} \Theta(n^2/\mathcal{B}) & \text{if } n^2 < c\mathcal{M} \text{ for suff. small const } c \leq 1, \\ 8Q(n/2) + 1 & \text{otherwise.} \end{cases}$$



Cache-Oblivious Algorithms

- **Cache-oblivious algorithms** [FLPR99]
 - No voodoo tuning parameters.
 - No explicit knowledge of caches.
 - Passively autotune.
 - Handle multilevel caches automatically.
 - Good in multitenancy environments.